第 12 讲：滑动窗口——维护一个连续区间的状态

0. 本讲的核心问题

题目经常要求在数组或字符串的连续部分中寻找答案：

- 长度为 `k` 的连续子数组最大和；
- 不含重复字符的最长子串；
- 和至少为目标值的最短子数组；
- 字符串中是否存在某种排列；
- 最小覆盖子串。

暴力方法通常枚举每个左端点、每个右端点，再重新计算区间内容，可能达到 `O(n²)` 或 `O(n³)`。

滑动窗口的核心思想是：

> 当区间边界只向前移动时，复用上一个区间的统计结果，而不是从头计算。

窗口通常表示半开区间：

```
[left, right]
```

在代码中也常把 `right` 作为当前刚加入的位置。

---

1. 什么题可能使用滑动窗口

看到以下信号时可以考虑：

1. 研究对象是连续子数组或连续子串；
2. 右端加入一个元素后，窗口状态能增量更新；
3. 左端删除一个元素后，窗口状态也能增量更新；
4. 存在某种单调性，使左端不需要回退。

不是所有连续区间题都能使用普通滑动窗口。

例如数组含负数时，“窗口和太大就缩小”可能不再安全，因为删除一个负数反而会使和增大。

---

2. 固定长度窗口

题目：求长度恰好为 `k` 的连续子数组最大和。

2.1 暴力方法

每个窗口重新求和：

```
[python]
def max_sum_fixed_brute(nums, k):
    best = float("-inf")
```

```
        for left in range(len(nums) - k + 1):
            current = sum(nums[left:left + k])
            best = max(best, current)

        return best
```

有约 `n` 个窗口，每次求和 `O(k)`：

```
O(nk)
```

2.2 复用前一个窗口

窗口向右移动一格：

```
   +
```

```
[python]
def max_sum_fixed(nums, k):
    if k <= 0 or k > len(nums):
        return None

    window_sum = sum(nums[:k])
    best = window_sum

    for right in range(k, len(nums)):
        entering = nums[right]
        leaving = nums[right - k]
        window_sum += entering - leaving
        best = max(best, window_sum)

    return best
```

时间降为 `O(n)`。

循环不变量：

> 更新后，`window_sum` 等于长度为 `k`、右端点为 `right` 的窗口元素之和。

---

3. 可变长度窗口的标准流程

可变窗口通常让右端不断扩张；当窗口不满足条件时，移动左端收缩。

```
[python]
left = 0

for right in range(len(data)):
    add(data[right])

    while window_is_invalid():
        remove(data[left])
        left += 1

    update_answer(left, right)
```

四步含义：

```
1. right
2.
3.         left
4.
```

3.1 为什么看似嵌套仍是 `O(n)`

`right` 从左到右最多移动 `n` 次；`left` 也只向右最多移动 `n` 次。

所有 `while` 的总执行次数最多为 `n`，因此通常是 `O(n)`，不是 `O(n²)`。

---

4. 核心题：无重复字符的最长子串

窗口内不允许出现重复字符。

4.1 使用集合

```
[python]
def length_of_longest_substring(text):
    left = 0
    window = set()
    best = 0

    for right, ch in enumerate(text):
        while ch in window:
            window.remove(text[left])
            left += 1

        window.add(ch)
        best = max(best, right - left + 1)

    return best
```

4.2 窗口不变量

在更新答案时：

> `window` 恰好包含 `text[left:right+1]` 中的字符，并且这些字符互不重复。

4.3 为什么必须使用 `while`

例如：

```
abba
```

加入第二个 `b` 时，可能需要连续移除多个左侧字符，直到旧 `b` 被移出窗口。

只写 `if` 不能保证窗口恢复合法。

---

5. 用最后出现位置直接跳跃

集合写法让 `left` 一步步移动。也可以记录每个字符最后出现的下标：

```
[python]
def length_of_longest_substring(text):
    last_index = {}
    left = 0
    best = 0

    for right, ch in enumerate(text):
        if ch in last_index and last_index[ch] >= left:
            left = last_index[ch] + 1

        last_index[ch] = right
        best = max(best, right - left + 1)

    return best
```

为什么要检查：

```
[python]
last_index[ch] >= left
```

因为字符的上一次出现可能已经在当前窗口左侧，不能让 `left` 倒退。

也可写成：

```
[python]
left = max(left, last_index[ch] + 1)
```

窗口左端必须单调向右。

---

6. 最短子数组：和至少为 target

假设 `nums` 中都是正整数。

当窗口和达到目标后，继续扩大只会更长；应该尽量从左侧缩小：

```
[python]
def min_subarray_len(target, nums):
    left = 0
    window_sum = 0
    best = float("inf")

    for right, x in enumerate(nums):
        window_sum += x

        while window_sum >= target:
            best = min(best, right - left + 1)
            window_sum -= nums[left]
            left += 1

    return 0 if best == float("inf") else best
```

6.1 为什么在收缩前更新答案

当前窗口已经满足条件，所以它是一个候选答案。记录后再删除左端，尝试得到更短的合法窗口。

6.2 为什么要求正数

正数保证：

- 扩张右端时，窗口和不会减小；
- 收缩左端时，窗口和不会增大。

这形成了单调性。

如果含负数，简单的左右滑动不能保证不漏答案，可能需要前缀和与单调队列等方法。

---

7. “最长合法”与“最短满足”的更新位置

这两类模板容易混淆。

7.1 最长合法窗口

先收缩到合法，再更新：

```
[python]
for right in range(n):
    add(right)

    while invalid():
```

```
            remove(left)
            left += 1

        best = max(best, right - left + 1)
```

例：最长无重复子串。

7.2 最短满足条件窗口

一旦满足，就在收缩过程中不断更新：

```
[python]
for right in range(n):
    add(right)

    while valid():
        best = min(best, right - left + 1)
        remove(left)
        left += 1
```

例：和至少为 target 的最短正数子数组。

不要只背代码，要先判断答案是在“合法状态”更新，还是在“满足状态下尽量收缩”时更新。

---

8. 固定长度频率窗口：找到所有字母异位词

给定 `text` 和较短字符串 `pattern`，找出 `text` 中所有与 `pattern` 字母频率相同的子串起点。

窗口长度固定为 `len(pattern)`。

```
[python]
def find_anagrams(text, pattern):
    if len(pattern) > len(text):
        return []

    need = {}
    window = {}

    for ch in pattern:
        need[ch] = need.get(ch, 0) + 1

    result = []
    left = 0

    for right, ch in enumerate(text):
        window[ch] = window.get(ch, 0) + 1

        if right - left + 1 > len(pattern):
            leaving = text[left]
            window[leaving] -= 1

            if window[leaving] == 0:
                del window[leaving]

            left += 1

        if window == need:
            result.append(left)

    return result
```

字典直接比较清晰，但每次比较可能与不同字符数量有关。字符集固定时，可以使用长度 26 的数组，或维护匹配种类数量进行优化。

---

9. 频率差与满足种类数

更复杂的字符串窗口通常维护：

```
need[ch]         ch
window[ch]          ch
formed
required
```

当某字符数量第一次达到要求：

```
[python]
if window[ch] == need[ch]:
    formed += 1
```

当移除字符导致它从满足变为不足：

```
[python]
if window[ch] == need[ch]:
    formed -= 1

window[ch] -= 1
```

注意减少 `formed` 的判断发生在计数减一之前。

窗口合法条件：

```
[python]
formed == required
```

这比每次完整比较两个字典更高效。

---

10. 进阶题：最小覆盖子串

目标：找到 `text` 中包含 `target` 所有字符及其数量的最短子串。

```
[python]
def min_window(text, target):
    if not target or len(target) > len(text):
        return ""

    need = {}

    for ch in target:
        need[ch] = need.get(ch, 0) + 1

    window = {}
    required = len(need)
    formed = 0
    left = 0
    best_length = float("inf")
    best_left = 0

    for right, ch in enumerate(text):
        window[ch] = window.get(ch, 0) + 1

        if ch in need and window[ch] == need[ch]:
            formed += 1

        while formed == required:
            length = right - left + 1

            if length < best_length:
                best_length = length
                best_left = left

            leaving = text[left]
```

```
                if leaving in need and window[leaving] == need[leaving]:
                    formed -= 1

                window[leaving] -= 1
                left += 1

        if best_length == float("inf"):
            return ""

        return text[best_left:best_left + best_length]
```

10.1 思考顺序

这是“最短满足条件”模板的典型应用。

---

11. 窗口状态应该保存什么

窗口不一定保存窗口元素本身，因为原数组已经保存了元素。额外状态通常是：

- 元素和 `window_sum`；
- 字符集合 `window`；
- 频率字典 `counts`；
- 不同元素数量 `distinct`；
- 最大值或最小值的候选结构；
- 满足要求的字符种类数 `formed`。

选择最小够用的状态。

例如只关心是否重复时，集合足够；关心重复次数时，需要频率字典。

---

12. 滑动窗口何时失效

12.1 缺少单调性

若数组含正负数，窗口和随扩张和收缩都可能增减。根据当前和无法唯一决定移动哪一侧。

12.2 研究对象不是连续区间

子序列允许跳过元素，通常不是普通滑动窗口问题。

12.3 左端需要回退

标准窗口要求左右边界只向前。如果正确算法需要让左端反复回退，通常需要其他方法。

12.4 无法快速更新状态

如果加入和删除一个元素后都无法高效维护条件，滑动窗口可能没有优势，或需要更高级的数据结构。

---

13. 常见错误

13.1 混淆子串、子数组和子序列

- 子串/子数组必须连续；
- 子序列可以跳过元素。

普通滑动窗口处理连续区间。

13.2 窗口长度少写或多写 1

闭区间 `[left, right]` 的长度是：

```
[python]
right - left + 1
```

13.3 忘记移除左端对状态的影响

移动 `left` 前要先从和、集合或计数中删除 `data[left]`。

13.4 应用 `if` 而不是 `while`

一次收缩不一定能恢复合法。只要条件仍不满足，就要继续收缩。

13.5 `left` 倒退

使用最后出现位置跳跃时：

```
[python]
left = max(left, last_index[ch] + 1)
```

13.6 在错误位置更新答案

先判断是“最长合法”还是“最短满足”，再决定答案更新在收缩前、收缩中还是收缩后。

13.7 删除计数为零的键遗漏

如果使用字典整体相等来判断，零计数键仍会影响字典比较，需要删除。

---

14. 本讲检查表

```
1.
2.
3. left right
4.
5.
6.
7.         left    if    while
8.
9.
10.
11.    set
12.      k
```

15. 本讲总结

1. 滑动窗口复用相邻连续区间的统计状态。
2. 固定窗口每次移除左端、加入右端。

3. 可变窗口通常由右端扩张，并用左端恢复合法性或寻找最短答案。
4. 两个指针各自只向前，因此嵌套 `while` 的总时间通常仍为 `O(n)`。
5. “最长合法”和“最短满足”有不同的答案更新位置。
6. 使用滑动窗口前必须确认连续性、可增量更新和边界移动的单调性。